Python Developer Syllabus

End-to-End Career Roadmap · From Zero to Job-Ready

6 Phases ~6–9 Months 50+ Projects 5 Career Paths

1	2	3	4	5	6
Foundations	Intermediate	Web Dev	Data & Auto	DevOps	Advanced
4–6 wks	*4–6 wks*	*6–8 wks*	*4–6 wks*	*4–5 wks*	*Ongoing*

Phase 1 — Python Foundations

⏱ 4–6 weeks

Build rock-solid Python fundamentals before touching any framework or library.

ENVIRONMENT SETUP

- Python installation & version management (pyenv)
- VS Code / PyCharm setup
- Virtual environments (venv, conda)
- pip & package management
- Git & GitHub basics

CORE LANGUAGE

- Data types: int, float, str, bool, None
- Variables, operators, expressions
- Control flow: if/elif/else, loops
- Functions: args, kwargs, default params
- Scope, closures, lambda functions

DATA STRUCTURES

- Lists, tuples, sets, dicts
- List & dict comprehensions
- Slicing, indexing, unpacking
- Built-in: map, filter, zip, enumerate
- Nested structures & iteration

OOP BASICS

- Classes & objects
- Constructors (__init__)
- Instance vs class variables
- Inheritance & polymorphism
- Dunder/magic methods

ERROR HANDLING & FILES

- try/except/finally blocks
- Custom exceptions
- File I/O (open, read, write)
- Context managers (with statement)
- JSON & CSV handling

🔨 Calculator CLI app | 🔨 Student grade manager | 🔨 File organizer script

⚙️ Phase 2 — Intermediate Python

⏱ 4–6 weeks

Level up with advanced patterns and the standard library that power real-world code.

ADVANCED FUNCTIONS

- Decorators & wrappers
- Generators & yield
- Itertools & functools
- Type hints & annotations
- Walrus operator & modern syntax

MODULES & PACKAGES

- Creating packages (__init__.py)
- Relative vs absolute imports
- Standard library: os, sys, pathlib, datetime, re
- Third-party packages with pip
- Publishing to PyPI basics

CONCURRENCY

- Threading vs multiprocessing
- asyncio & async/await
- aiohttp for async HTTP
- concurrent.futures
- GIL and when to use each

TESTING

- unittest & pytest
- Fixtures and parameterized tests
- Mocking (unittest.mock)
- Code coverage (pytest-cov)
- TDD workflow

DATA HANDLING

- Regular expressions (re module)
- XML & HTML parsing (BeautifulSoup)
- Working with APIs (requests)
- Environment variables (dotenv)
- Logging best practices

🔨 Web scraper | 🔨 CLI expense tracker (JSON) | 🔨 Async news aggregator

🌐 Phase 3 — Web Development

⏱ 6–8 weeks

Build production-grade web applications and REST APIs using Python's leading frameworks.

FASTAPI (RECOMMENDED)

- Path operations & routers
- Pydantic models & validation
- Dependency injection
- OAuth2 & JWT auth
- Background tasks & middleware

DJANGO (FULL-STACK)

- MVT architecture
- ORM: models, queries, migrations
- Django REST Framework (DRF)
- Admin panel customization
- Django signals & celery

DATABASES

- SQL fundamentals (PostgreSQL/MySQL)
- SQLAlchemy ORM & core
- Alembic migrations
- Redis for caching & sessions
- Database indexing & optimization

API DESIGN

- REST principles & HTTP methods
- OpenAPI / Swagger docs
- Versioning strategies
- Rate limiting & throttling
- Error handling conventions

FRONTEND INTEGRATION

- CORS configuration
- Template rendering (Jinja2)
- WebSockets
- File uploads & static files
- SSE (Server-Sent Events)

🔨 Blog API + Auth (FastAPI) | 🔨 E-commerce REST API | 🔨 Real-time chat (WebSockets)

📊 Phase 4 — Data & Automation

⏱ 4–6 weeks

Python's killer domain — from data wrangling and visualization to task automation.

DATA SCIENCE STACK

- NumPy: arrays & vectorized ops
- Pandas: DataFrames, groupby, merge
- Matplotlib & Seaborn visualization
- Jupyter Notebooks workflow
- Data cleaning & EDA pipelines

AUTOMATION

- Selenium & Playwright (browser)
- PyAutoGUI (desktop automation)
- Scheduled tasks (APScheduler, cron)
- Email automation (smtplib, SendGrid)
- PDF & Excel manipulation (openpyxl)

APIS & INTEGRATIONS

- REST API consumption patterns
- Webhooks
- Google Sheets & Drive API
- Telegram / WhatsApp bots
- Zerodha Kite Connect (trading APIs)

🔨 Stock market dashboard | 🔨 Automated email reports | 🔨 OI-based trading scanner

🚀 Phase 5 — DevOps & Deployment

⏱ 4–5 weeks

Ship code like a professional — containerize, automate, monitor, and scale.

DOCKER & CONTAINERS

- Dockerfile creation
- docker-compose for multi-service apps
- Image layers & optimization
- Container registries (Docker Hub, ECR)
- Environment variables in containers

CI/CD

- GitHub Actions workflows
- Running tests in CI
- Automated Docker builds
- Deployment pipelines
- Secrets management

CLOUD DEPLOYMENT

- AWS / GCP / Azure basics
- EC2, S3, RDS overview
- Serverless: AWS Lambda + API Gateway
- Railway, Render, Fly.io (PaaS)
- Nginx & Gunicorn setup

MONITORING & LOGGING

- Structured logging (loguru)
- Sentry for error tracking
- Prometheus + Grafana basics
- Health checks & uptime monitoring
- Performance profiling (cProfile)

🔨 Dockerize FastAPI app to Render

🔨 GitHub Actions CI/CD pipeline

🔨 Deploy on AWS EC2

🧠 Phase 6 — Advanced & Specializations

⏱ Ongoing

Pick a domain mastery path aligned with your career goal and go deep.

MACHINE LEARNING PATH

- Scikit-learn: regression, classification
- Feature engineering & pipelines
- Model evaluation & cross-validation
- XGBoost & LightGBM
- MLflow for experiment tracking

AI / LLM PATH

- OpenAI / Anthropic API integration
- LangChain & LlamaIndex
- RAG (Retrieval-Augmented Generation)
- Vector databases (Pinecone, ChromaDB)
- Prompt engineering & fine-tuning

SYSTEM DESIGN

- Microservices architecture
- Message queues (Celery + Redis)
- Caching strategies
- Horizontal scaling patterns
- API gateway patterns

CODE QUALITY & ARCHITECTURE

- Design patterns (Factory, Singleton)
- SOLID principles
- Clean / Hexagonal architecture
- Code review practices
- Performance optimization

🔨 ML pipeline with MLflow | 🔨 AI chatbot with RAG | 🔨 Microservices + message queue

CAREER PATHS AFTER COMPLETION

🖥️

Backend Developer

· FastAPI / Django
· PostgreSQL
· Docker
· REST APIs
· Redis

🔧

Data Engineer

· Pandas
· SQL
· Airflow
· Spark
· Cloud (AWS/GCP)

🤖

ML Engineer

· Scikit-learn
· PyTorch
· MLflow
· Docker
· Cloud ML

✨

AI/LLM Developer

· LangChain
· RAG
· Vector DBs
· APIs
· Prompt Eng.

⚡

Automation Eng.

· Selenium
· Scripting
· APIs
· Scheduling
· Bots

CAREER-BUILDING TIPS

- ✓ Build in public — GitHub commits daily
- ✓ Write about what you learn (blog/LinkedIn)
- ✓ Contribute to open source from Phase 3
- ✓ Solve 1 LeetCode/HackerRank problem daily
- ✓ Network in Python communities (PyPI, Discord)
- ✓ Create a portfolio with 3–5 real projects
- ✓ Learn SQL deeply — it's always needed
- ✓ Apply to jobs from Phase 3 onward